SmolVLA — Gesture Mimic Optimization

From 2 inf/s to 13–15 inf/s | MAE 6 → 2.7

PAVS

Optimization Sequence — Throughput & Accuracy Journey

Step-by-step changes on SmolVLA + LeRobot (CPU / Strix)

#	Step	What changed	Throughput	Track
1	**Diffusion steps ↓**	Reduced denoising steps **30 → 2** (extra steps shown unnecessary)	2 → **4.5–5 inf/s**	Throughput
2	**torch.compile**	Rewrote LeRobot SmolVLA modules so the full flow forms a single graph → optimized CPU kernels	→ **8.5 inf/s**	Throughput
3	**fp32 → fp16**	Cast weights to fp16 *before* compile so the graph has no up/down-scale ops around each layer	→ **13–15 inf/s**	Throughput
4	**RTC on (Real-Time Chunking)**	Camera pipeline + model run on parallel threads sharing the GPU → reactive end-to-end pipeline	Reactivity ↑	Throughput
5	**chunk_size_threshold ↓**	Next inference triggered after only **5** consumed actions → fresher images drive predictions	Reactivity ↑	Throughput
6	**Action smoothing**	Post-processing removes jitter/shake in continuous-action output	Smoothness ↑	Accuracy
7	**More data + fine-tune**	Scaled training data and fine-tuned model	MAE **6 → 2.7**	Accuracy

Bottom line: ~7× throughput (2 → 13–15 inf/s) and **~55% lower MAE** (6 → 2.7) — model is now usable for live gesture mimic on Strix.

Why Throughput Matters — Track 1

Speed is not just speed; it directly drives behavior quality

What each speed step bought us

- **Diffusion 30 → 2:** removed redundant compute — denoising had diminishing returns past 2 steps.
- **torch.compile:** fused operators into a single graph → CPU kernels run optimized, no Python overhead per layer.
- **fp16 (compile-aware):** halved memory & compute. Casting weights *before* compile is critical(tricky wrt rocm+torch.complie) — otherwise compile inserts upscale / downscale around every layer and the precision win is eaten by overhead.
- **RTC + lower `chunk_size_threshold`:** model and camera no longer wait on each other; new frames trigger inference every 5 actions instead of waiting for a full chunk.

Throughput ladder

Stage	inf/s
Baseline	**2.0**
+ Diffusion steps 30 → 2	**4.5–5**
+ torch.compile (graph)	**8.5**
+ fp16 (compile-aware)	**13–15**
+ RTC + chunk_threshold = 5	Reactive pipeline

> **For management:** Each step is independent and stackable. Throughput is now **CPU-deployable in real time** — no GPU dependency, no model swap.

Why Throughput → Accuracy → Smoothness — Track 2

Faster model = more accurate, more responsive robot

Throughput → Accuracy (coupling)

- A slow model **drops frames** from the camera stream → loses sync between operator and robot mimic.
- A slow model must predict **far into the future** from stale observations — long-horizon predictions are inherently unreliable.
- A **fast** model consumes fresh frames and **overwrites** old predictions with new ones grounded in current state → less guessing, tighter operator ↔️ robot sync.

Smoothness (jitter removal)

- Continuous-action output + limited training data → visible shake/jitter on the robot.
- Added **post-processing smoothing** on the action stream → smooth, deployable motion without retraining.

Accuracy ladder (MAE, lower is better)

Stage	MAE
Past delivery model	**~6.0**
+ Combined dataset & fine-tune	**~2.7**
Target for usable mimic	**< 3.0**

Two-track summary

- **Throughput track:** diffusion ↓, compile, fp16, RTC, chunk threshold → **7× faster**
- **Accuracy track:** more data + fine-tune + action smoothing → **MAE 6 → 2.7**, jitter removed

> **Net result:** a real-time, smooth, accurate gesture-mimic stack on SmolVLA + LeRobot — ready for the next round of operator trials.